

Program Analysis 2025-26

Lecture #03

Hoare Logic



**Program correctness:
a long standing problem**

Origins? Turing's assertions

“how can one check a routine in the sense of making sure that it is right?”
Alan Turing (1949)



Friday, 24th June.

Checking a large routine. by Dr. A. Turing.

How can one check a routine in the sense of making sure that it is right?

In order that the man who checks may not have too difficult a task the programmer should make a number of definite assertions which can be checked individually, and from which the correctness of the whole programme easily follows.

Consider the analogy of checking an addition. If it is given as:

1374
5906
6719
4337
7768
—

26104

one must check the whole at one sitting, because of the carries.

But if the totals for the various columns are given, as below:

1374
5906
6719
4337
7768
—

3974
2213
—

26104

the checker's work is much easier being split up into the checking of the various assertions $3 + 9 + 7 + 3 + 7 = 29$ etc. and the small addition

3794
2213
—
26104

This principle can be applied to the process of checking a large routine but we will illustrate the method by means of a small routine viz. one to obtain n without the use of a multiplier, multiplication being carried out by repeated addition.

At a typical moment of the process we have recorded r and $s \cdot r$ for some r, s . We can change $s \cdot r$ to $(s+1) \cdot r$ by addition of r . When $s = r+1$ we can change r to $r+1$ by a transfer. Unfortunately there is no coding system sufficiently generally known to justify giving the routine for this process in full, but the flow diagram given in Fig.1 will be sufficient for illustration.

Each 'box' of the flow diagram represents a straight sequence of instructions without changes of control. The following convention is used:

(i) a dashed letter indicates the value at the end of the process represented by the box:

(ii) an undashed letter represents the initial value of a quantity.

One cannot equate similar letters appearing in different boxes, but it is intended that the following identifications be valid throughout

Checking factorial

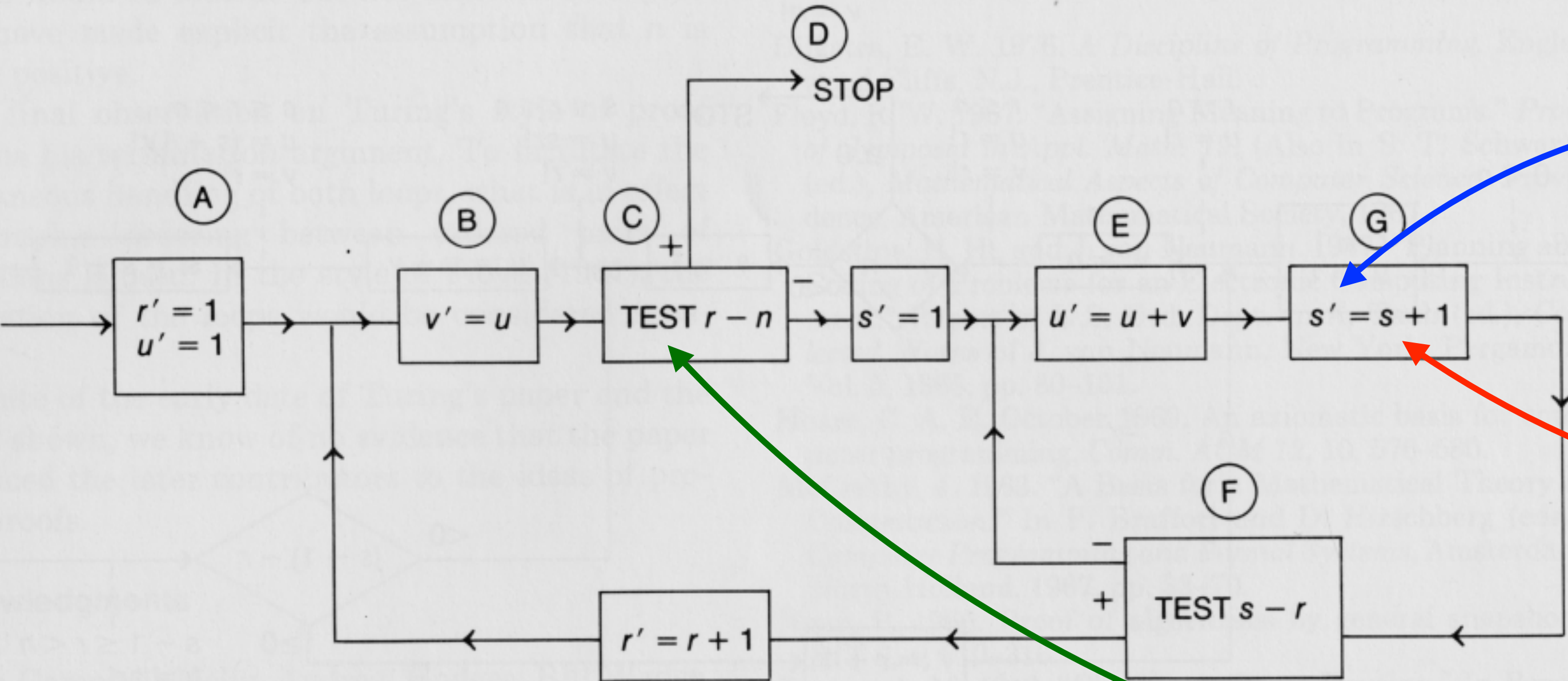


Figure 1 (Redrawn from Turing's original)

STORAGE LOCATION	(INITIAL) Ⓐ k = 6	Ⓑ k = 5	Ⓒ k = 4	(STOP) Ⓓ k = 0	Ⓔ k = 3	Ⓕ k = 1	Ⓖ k = 2
27 s					s	s + 1	s
28 r		r	r	n	r	r	r
29 n	n	n	n		n	n	n
30 u		⌊r	⌊r		s⌊r	(s + 1)⌊r	(s + 1)⌊r
31 v		⌊r	⌊r	⌊n	⌊r	⌊r	⌊r
	TO Ⓑ [Ⓐ] WITH r' = 1 u' = 1	TO Ⓒ	TO Ⓓ IF r = n TO Ⓔ IF r < n		TO Ⓖ WITH r' = r + 1 IF s ≥ r TO Ⓔ WITH s' = s + 1 IF s < r		TO Ⓕ

Figure 2 (Redrawn from Turing's original)

- a dashed letter indicates the value at the end of the process represented by the box
- an undashed letter represents the initial value of a quantity
- TEST is test for zero
- ⌊ denotes factorial
- at the end (D) v = n!

General snapshots (P. Naur, 1966)

BIT 6 (1966), 310–316

PROOF OF ALGORITHMS BY GENERAL SNAPSHOTS

PETER NAUR

Abstract.

A constructive approach to the question of proofs of algorithms is to consider proofs that an object resulting from the execution of an algorithm possesses certain static characteristics. It is shown by an elementary example how this possibility may be used to prove the correctness of an algorithm written in ALGOL 60. The stepping stone of the approach is what is called General Snapshots, i.e. expressions of static conditions existing whenever the execution of the algorithm reaches particular points. General Snapshots are further shown to be useful for constructing algorithms.

Key words: Algorithm, proof, computer, programming.

Introduction.

It is a deplorable consequence of the lack of influence of mathematical thinking on the way in which computer programming is currently being pursued, that the regular use of systematic proof procedures, or even the realization that such proof procedures exist, is unknown to the large majority of programmers. Undoubtedly, this fact accounts for at least a large share of the unreliability and the attendant lack of over-all effectiveness of programs as they are used to-day.

Historically this state of affairs is easily explained. Large scale computer programming started so recently that all of its practitioners are, in fact, amateurs. At the same time the modern computers are so effective that they offer advantages in use even when their powers are largely wasted. The stress has been on always larger, and, allegedly, more powerful systems, in spite of the fact that the available programmer competence often is unable to cope with their complexities.

However, a reaction is bound to come. We cannot indefinitely continue to build on sand. When this is realized there will be an increased interest in the less glamorous, but more solid, basic principles. This will go in parallel with the introduction of these principles in the elementary school curricula. One subject which will then come up for attention is that of proving the correctness of algorithms. The purpose of the present article is to show in an elementary way that this subject not only exists, but is ripe to be used in practise. The illustrations are phrased in ALGOL 60, but the technique may be used with any programming language.

Copyright © 1966 by Peter Naur.

“expression of static conditions existing whenever the execution of the algorithm reaches particular points”

Greatest number, with snapshots

comment General Snapshot 1: $1 \leq N$;

$r := 1$;

comment General Snapshot 2: $1 \leq N, r = 1$;

for $i := 2$ **step** 1 **until** N **do**

begin *comment General Snapshot 3: $2 \leq i \leq N, 1 \leq r \leq i - 1$,*

$A[r]$ is the greatest among the elements $A[1], A[2], \dots, A[i - 1]$;

if $A[i] > A[r]$ **then** $r := i$;

comment General Snapshot 4: $2 \leq i \leq N, 1 \leq r \leq i, A[r]$ is the greatest among the elements $A[1], A[2], \dots, A[i]$;

end;

comment General Snapshot 5: $1 \leq r \leq N, A[r]$ is the greatest among the elements $A[1], A[2], \dots, A[N]$;

$R := A[r]$;

comment General Snapshot 6: R is the greatest value of any element, $A[1], A[2], \dots, A[N]$;

Floyd's interpretations (1967)

Robert W. Floyd

ASSIGNING MEANINGS TO PROGRAMS¹

Introduction. This paper attempts to provide an adequate basis for formal definitions of the meanings of programs in appropriately defined programming languages, in such a way that a rigorous standard is established for proofs about computer programs, including proofs of correctness, equivalence, and termination. The basis of our approach is the notion of an interpretation of a program: that is, an association of a proposition with each connection in the flow of control through a program, where the proposition is asserted to hold whenever that connection is taken. To prevent an interpretation from being chosen arbitrarily, a condition is imposed on each command of the program. This condition guarantees that whenever a command is reached by way of a connection whose associated proposition is then true, it will be left (if at all) by a connection whose associated proposition will be true at that time. Then by induction on the number of commands executed, one sees that if a program is entered by a connection whose associated proposition is then true, it will be left (if at all) by a connection whose associated proposition will be true at that time. By this means, we may prove certain properties of programs, particularly properties of the form: "If the initial values of the program variables satisfy the relation R_1 , the final values on completion will satisfy the relation R_2 ." Proofs of termination are dealt with by showing that each step of a program decreases some entity which cannot decrease indefinitely.

These modes of proof of correctness and termination are not original; they are based on ideas of Perlis and Gorn, and may have made their earliest appearance in an unpublished paper by Gorn. The establishment of formal standards for proofs about programs in languages which admit assignments, transfer of control, etc., and the proposal that the semantics of a programming language may be defined independently of all processors for that language, by establishing standards of rigor for proofs about

¹This work was supported by the Advanced Research Projects Agency of the Office of the Secretary of Defense (SD-146).

“an association of a proposition with each connection in the flow of control through a program, where the proposition is asserted to hold whenever that connection is taken”

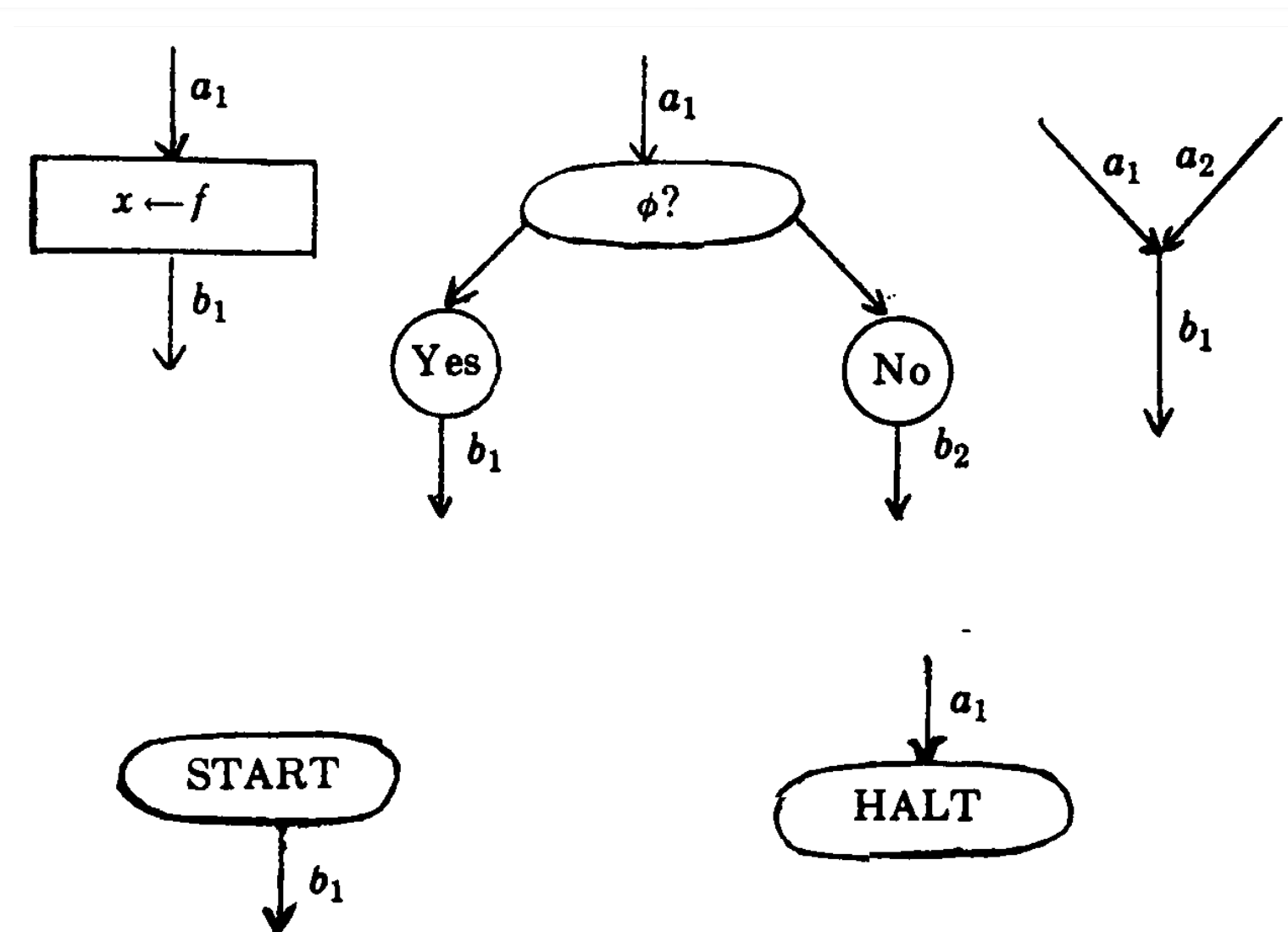


FIGURE 2



Floyd's examples

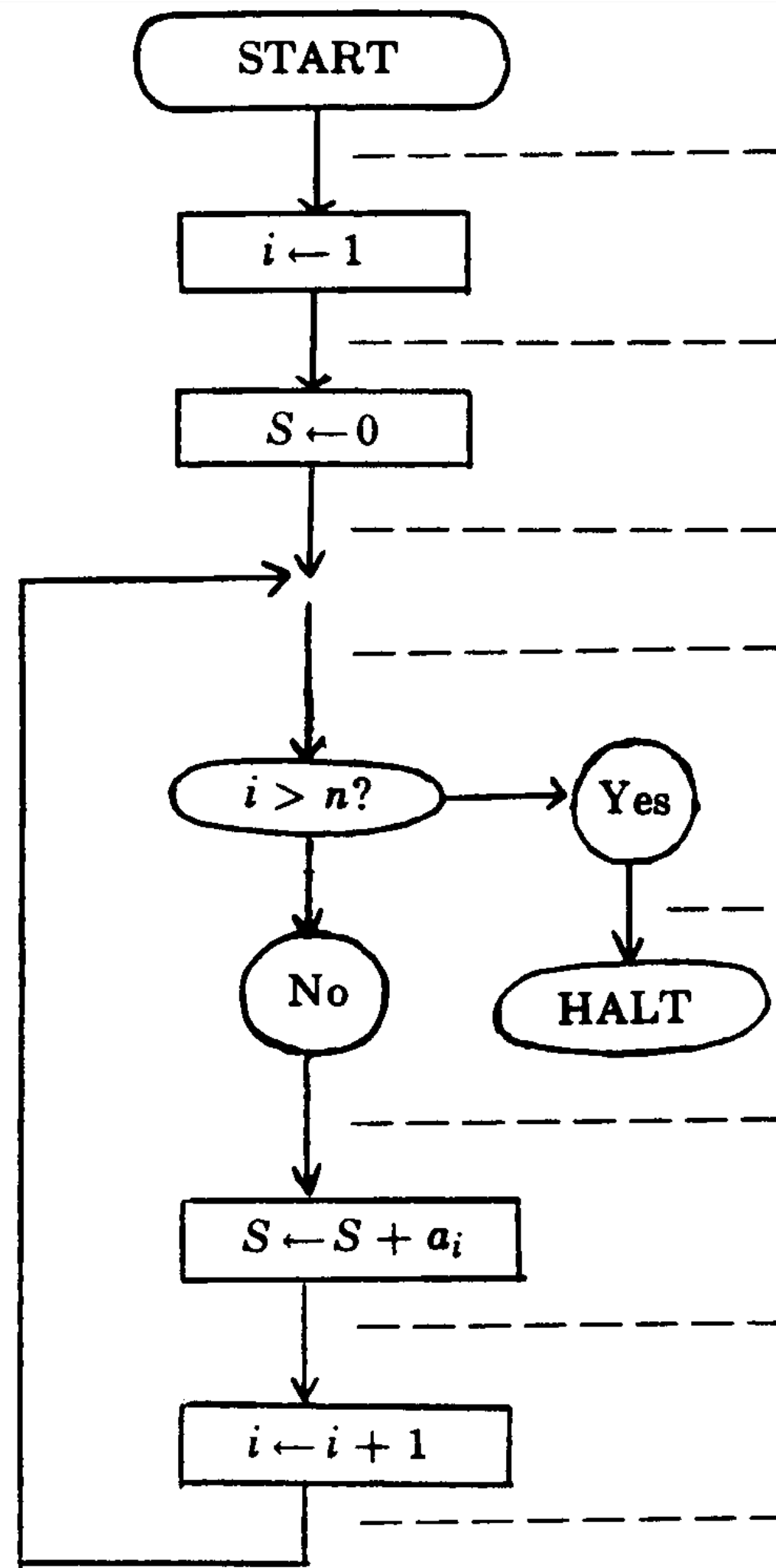


FIGURE 1. Flowchart of program to compute $S = \sum_{j=1}^n a_j$ ($n \geq 0$)

Floyd's examples

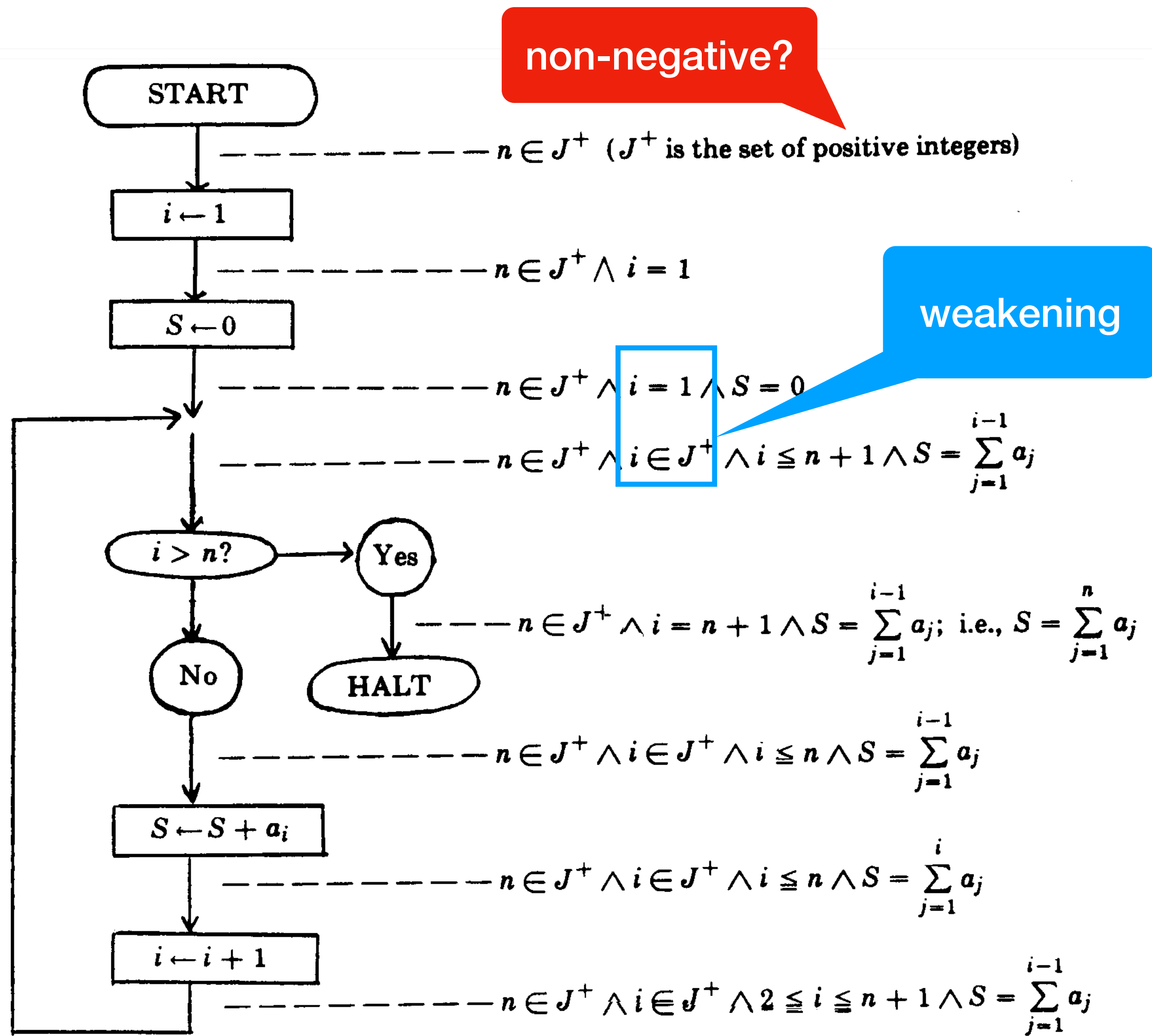


FIGURE 1. Flowchart of program to compute $S = \sum_{j=1}^n a_j$ ($n \geq 0$)

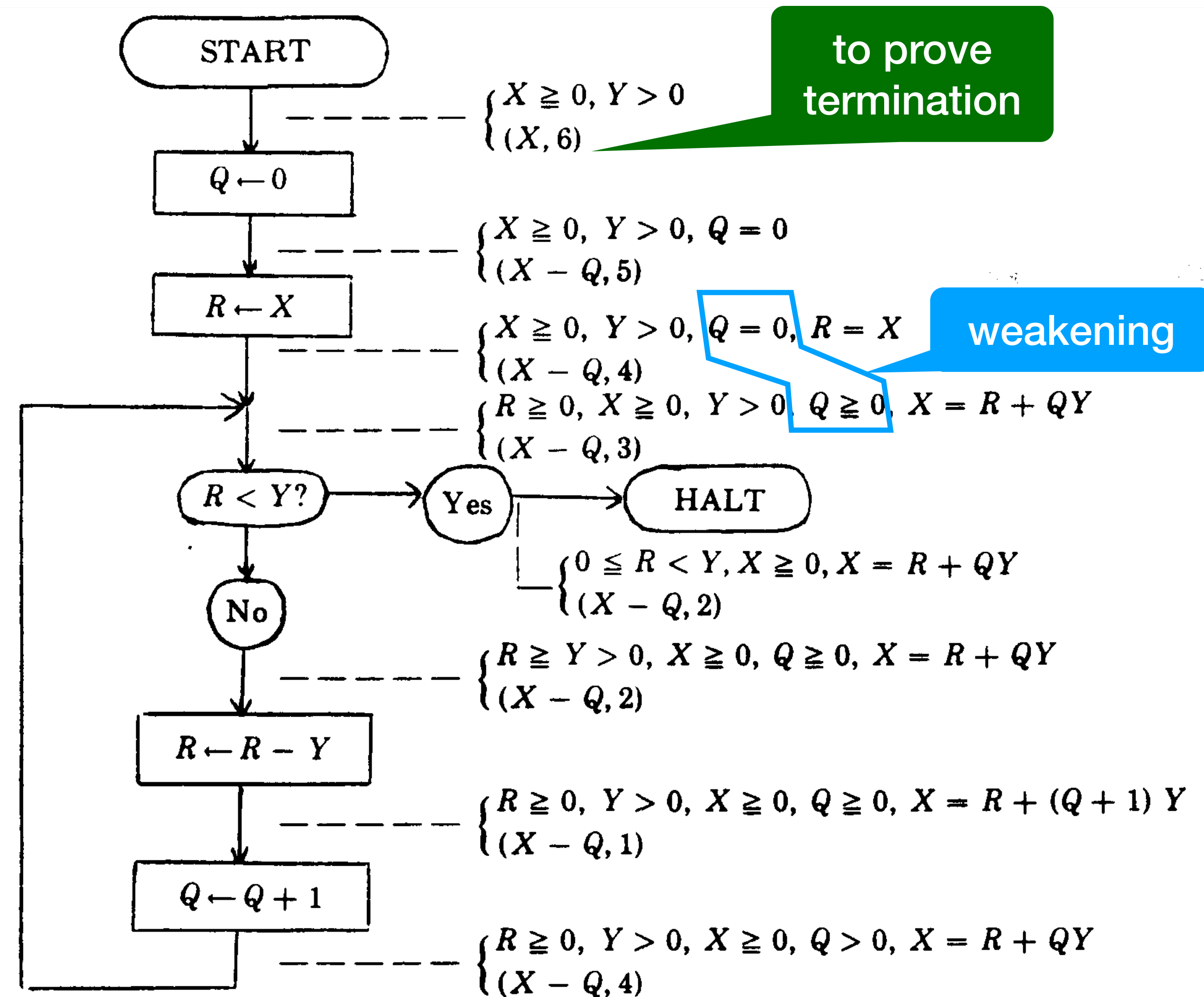


FIGURE 5. Algorithm to compute quotient Q and remainder R of $X \div Y$, for integers $X \geq 0, Y > 0$

Hoare Logic

An Axiomatic Basis for Computer Programming

C. A. R. HOARE
The Queen's University of Belfast, Northern Ireland*

In this paper an attempt is made to explore the logical foundations of computer programming by use of techniques which were first applied in the study of geometry and have later been extended to other branches of mathematics. This involves the elucidation of sets of axioms and rules of inference which can be used in proofs of the properties of computer programs. Examples are given of such axioms and rules, and a formal proof of a simple theorem is displayed. Finally, it is argued that important advantages, both theoretical and practical, may follow from a pursuance of these topics.

KEY WORDS AND PHRASES: axiomatic method, theory of programming[†] proofs of programs, formal language definition, programming language design, machine-independent programming, program documentation
CR CATEGORY: 4.0, 4.21, 4.22, 5.20, 5.21, 5.23, 5.24

1. Introduction

Computer programming is an exact science in that all the properties of a program and all the consequences of executing it in any given environment can, in principle, be found out from the text of the program itself by means of purely deductive reasoning. Deductive reasoning involves the application of valid rules of inference to sets of valid axioms. It is therefore desirable and interesting to elucidate the axioms and rules of inference which underlie our reasoning about computer programs. The exact choice of axioms will to some extent depend on the choice of programming language. For illustrative purposes, this paper is confined to a very simple language, which is effectively a subset of all current procedure-oriented languages.

2. Computer Arithmetic

The first requirement in valid reasoning about a program is to know the properties of the elementary operations which it invokes, for example, addition and multiplication of integers. Unfortunately, in several respects computer arithmetic is not the same as the arithmetic familiar to mathematicians, and it is necessary to exercise some care in selecting an appropriate set of axioms. For example, the axioms displayed in Table I are rather a small selection of axioms relevant to integers. From this incomplete set

of axioms it is possible to deduce such simple theorems as:

$$\begin{aligned}x &= x + y \times 0 \\ y \leq r \supset r + y \times q &= (r - y) + y \times (1 + q) \\ \text{A5 } (r - y) + y \times (1 + q) &= (r - y) + (y \times 1 + y \times q) \\ \text{A9 } &= (r - y) + (y + y \times q) \\ \text{A3 } &= ((r - y) + y) + y \times q \\ \text{A6 } &= r + y \times q \quad \text{provided } y \leq r\end{aligned}$$

The axioms A1 to A9 are, of course, true of the traditional infinite set of integers in mathematics. However, they are also true of the finite sets of “integers” which are manipulated by computers provided that they are confined to *nonnegative* numbers. Their truth is independent of the size of the set; furthermore, it is largely independent of the choice of technique applied in the event of “overflow”; for example:

- (1) Strict interpretation: the result of an overflowing operation does not exist; when overflow occurs, the offending program never completes its operation. Note that in this case, the equalities of A1 to A9 are strict, in the sense that both sides exist or fail to exist together.
- (2) Firm boundary: the result of an overflowing operation is taken as the maximum value represented.
- (3) Modulo arithmetic: the result of an overflowing operation is computed modulo the size of the set of integers represented.

These three techniques are illustrated in Table II by addition and multiplication tables for a trivially small model in which 0, 1, 2, and 3 are the only integers represented.

It is interesting to note that the different systems satisfying axioms A1 to A9 may be rigorously distinguished from each other by choosing a particular one of a set of mutually exclusive supplementary axioms. For example, infinite arithmetic satisfies the axiom:

$$\text{A10}_I \quad \neg \exists x \forall y \quad (y \leq x),$$

where all finite arithmetics satisfy:

$$\text{A10}_F \quad \forall x \quad (x \leq \text{max})$$

where “max” denotes the largest integer represented. Similarly, the three treatments of overflow may be distinguished by a choice of one of the following axioms relating to the value of $\text{max} + 1$:

$$\text{A11}_S \quad \neg \exists x \quad (x = \text{max} + 1) \quad (\text{strict interpretation})$$

$$\text{A11}_F \quad \text{max} + 1 = \text{max} \quad (\text{firm boundary})$$

$$\text{A11}_M \quad \text{max} + 1 = 0 \quad (\text{modulo arithmetic})$$

Having selected one of these axioms, it is possible to use it in deducing the properties of programs; however,

* Department of Computer Science

“the purpose of this study is to provide a logical basis for proofs of the properties of a program”

C.A.R. Hoare (1969)



Hoare's example

find the quotient q and the remainder r obtained on dividing x by y

$((r := x; \quad q := 0); \quad \mathbf{while}$
 $\quad y \leq r \mathbf{do} (r := r - y; \quad q := 1 + q))$

$$\neg y \leq r \wedge x = r + y \times q$$

TABLE III

Line number	Formal proof	Justification
1	true $\supset x = x + y \times 0$	Lemma 1
2	$x = x + y \times 0 \{r := x\} x = r + y \times 0$	D0
3	$x = r + y \times 0 \{q := 0\} x = r + y \times q$	D0
4	true $\{r := x\} x = r + y \times 0$	D1 (1, 2)
5	true $\{r := x; \quad q := 0\} x = r + y \times q$	D2 (4, 3)
6	$x = r + y \times q \wedge y \leq r \supset x =$ $(r - y) + y \times (1 + q)$	Lemma 2
7	$x = (r - y) + y \times (1 + q) \{r := r - y\} x =$ $r + y \times (1 + q)$	D0
8	$x = r + y \times (1 + q) \{q := 1 + q\} x =$ $r + y \times q$	D0
9	$x = (r - y) + y \times (1 + q) \{r := r - y;$ $\quad q := 1 + q\} x = r + y \times q$	D2 (7, 8)
10	$x = r + y \times q \wedge y \leq r \{r := r - y;$ $\quad q := 1 + q\} x = r + y \times q$	D1 (6, 9)
11	$x = r + y \times q \{\mathbf{while} \ y \leq r \mathbf{do}$ $\quad (r := r - y; \quad q := 1 + q)\}$ $\neg y \leq r \wedge x = r + y \times q$	D3 (10)
12	true $\{((r := x; \quad q := 0); \quad \mathbf{while} \ y \leq r \mathbf{do}$ $\quad (r := r - y; \quad q := 1 + q))\} \neg y \leq r \wedge x =$ $r + y \times q$	D2 (5, 11)

NOTES

1. The left hand column is used to number the lines, and the right hand column to justify each line, by appealing to an axiom, a lemma or a rule of inference applied to one or two previous lines, indicated in brackets. Neither of these columns is part of the formal proof. For example, line 2 is an instance of the axiom of assignment (D0); line 12 is obtained from lines 5 and 11 by application of the rule of composition (D2).

2. Lemma 1 may be proved from axioms A7 and A8.

3. Lemma 2 follows directly from the theorem proved in Sec. 2.

Inference rules

$$\frac{\phi_1 \phi_2 \cdots \phi_n}{\phi}$$

Premises
Conclusion

- **Inference rules** are logical clauses formed by a set of premises and a conclusion
- Their function is to establish that, **if the premises are satisfied, then the conclusion hold**
- Rules without premises represent facts that are accepted without proof, called **axioms**

Proof system

A set of inference rules

$$\frac{}{\text{pos}(1)} \qquad \frac{\text{pos}(x) \quad \text{pos}(y)}{\text{pos}(x + y)}$$

Derivation Tree

$$\frac{[\text{base}] \quad \text{pos}(1)}{\text{pos}(1)} \quad \frac{\text{pos}(x) \text{ pos}(y)}{\text{pos}(x + y)} [\text{sum}]$$

Inference rules allow **proof** to be constructed in the form of **derivation trees**, tree structures that represent the logical path by which a conclusion is reached starting from a set of premises

$$\frac{\frac{\frac{\text{pos}(1)}{\text{pos}(1)} \quad \frac{\text{pos}(1)}{\text{pos}(1)}}{\text{pos}(2)} \quad \frac{\frac{\text{pos}(1)}{\text{pos}(1)} \quad \frac{\text{pos}(1)}{\text{pos}(1)}}{\text{pos}(2)}}{\text{pos}(4)} \quad \frac{\text{pos}(1)}{\text{pos}(1)}}{\text{pos}(5)}$$

A simple imperative language

command

$$\begin{array}{l} c ::= \\ | \\ | \\ | \\ | \\ | \end{array} \begin{array}{l} x := a \\ \text{skip} \\ c_1; c_2 \\ \text{if } b \text{ then } c_1 \text{ else } c_2 \\ \text{while } b \text{ do } c \end{array}$$

integer
variable

arithmetic
expression

Boolean
expression

$$a ::= n \mid x \mid a_1 + a_2 \mid \dots$$
$$b ::= a_1 \leq a_2 \mid b_1 \wedge b_2 \mid \dots$$

Hoare Logic (HL)

Hoare triples

$$\{P\} \ c \ \{Q\}$$

Precondition

Postcondition

Whenever c is executed in a state satisfying P
if the execution of c terminates, it terminates in a state satisfying Q

Hoare triples

$$\{P\} c \{Q\}$$

iff

$$[[c]]P \subseteq Q$$

We talk about **partial correctness** because the previous semantic condition does not guarantee that the execution of c on states satisfying P terminate

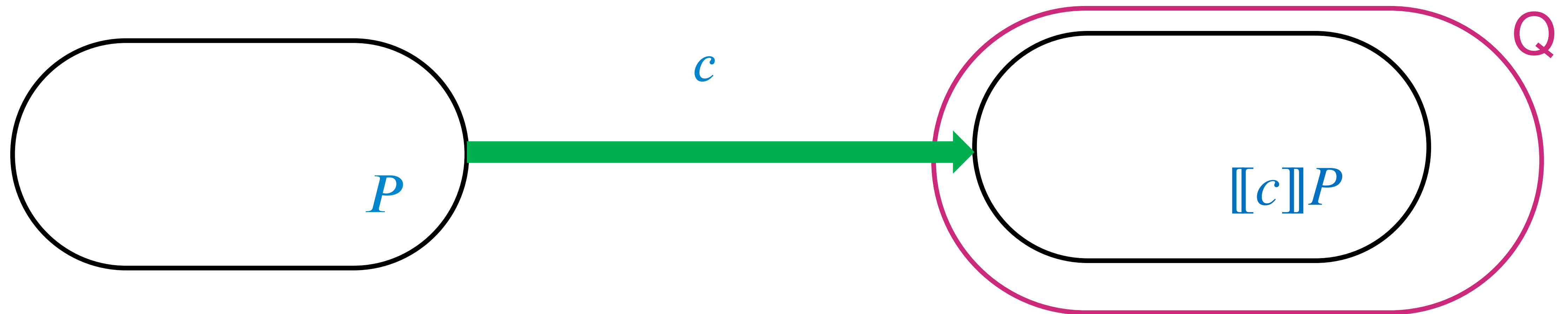
Hoare triples

$$\{P\} c \{Q\}$$

iff

$$\llbracket c \rrbracket P \subseteq Q$$

Q is an **over-approximation** of the states that can be reached by states in P

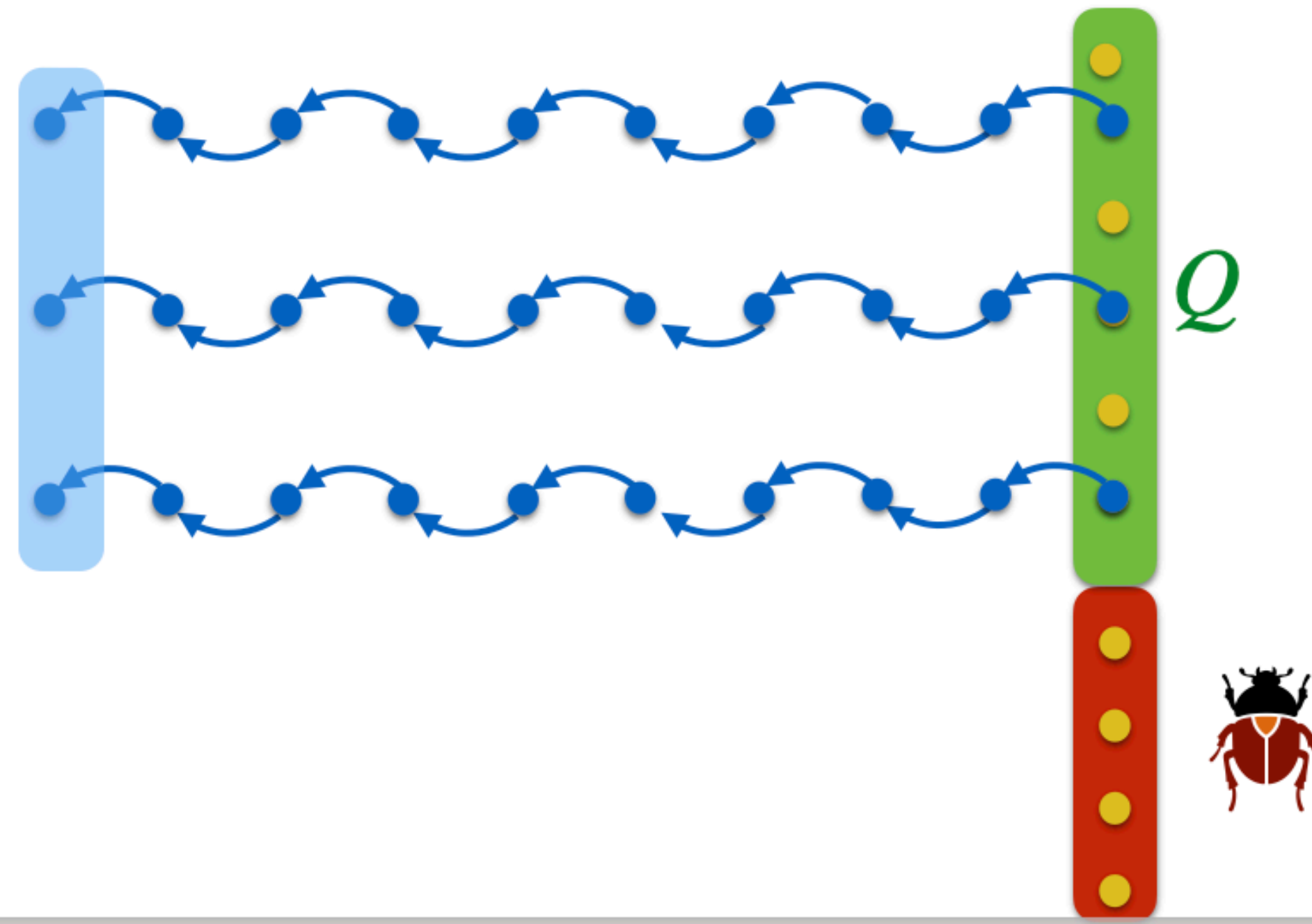


Hoare triples

$\{x < 0\} y := x * x \{y > 0\}$ $\{x < 0\} y := x * x \{y > 1\}$

$\{x < 0\} y := x * x \{y \geq 0\}$

$\{x < 0\} y := x * x \{y \geq -1\}$



Hoare Logic as a Proof System

- Programs are annotated with logical assertions
- Correctness is reduced to proving Hoare triples
- Proofs are derived using inference rules

Instead of verifying a **whole program at once**:

- We specify each command locally
- We assign:
 - a precondition describing the required input state
 - a postcondition describing the guaranteed output state
- We use syntactic proof rules for each construct

Hoare Logic as a Proof System

For every language construct, we define a rule:

- Assignment
- Sequential composition
- Conditional statements
- While loops

Each rule explains how to compute or transform:

Precondition



Postcondition

An obvious axiom

$$\frac{}{\{P\} \text{ skip } \{P\}}$$

$$\{x > 0\} \text{ skip } \{x > 0\}$$

Let's work it out together

$$\{P\} x := a ?$$

$$\{x > 0, y = 3x, z = x\} x := x + y ?$$

Floyd's axiom for assignment

$$\{P\} x := a \{ \exists x'. P[x'/x] \wedge x = a[x'/x] \}$$

syntax
replacement

syntax
replacement

$$\{\text{true}\} r := x \{ \exists r'. \text{true}, r = x \} \equiv \{r = x\}$$

$$\begin{aligned} \{x = r + qy\} r := r - y & \{ \exists r'. x = r' + qy, r = r' - y \} \\ & \equiv \{ \exists r'. x = r + y + qy, r' = r + y \} \\ & \equiv \{x = r + (q + 1)y\} \end{aligned}$$

Hoare's axiom for assignment

$$\frac{}{? \, x := a \, \{Q\}}$$

$$\{z > 0, x = z, y = 3z\} \, x := x + y \, \{z > 0, x = 4z, y = 3z\}$$

Hoare's axiom for assignment

$$\frac{}{\{Q[a/x]\} x := a \{Q\}}$$

syntax
replacement

$$\{\text{true}\} \equiv \{x = x + 0y\} \quad r := x \quad \{x = r + 0y\}$$

$$\{x = r\} \equiv \{x = r + 0y\} \quad q := 0 \quad \{x = r + qy\}$$

$$\{x = r + qy\} \equiv$$

$$\{x = r - y + (q + 1)y\} \quad r := r - y \quad \{x = r + (q + 1)y\}$$

An observation

forward oriented

[Floyd's]

$$\{P\} x := a \{ \exists x'. P[x'/x] \wedge x = a[x'/x] \}$$

backward oriented

[Hoare's]

$$\{Q[a/x]\} x := a \{Q\}$$

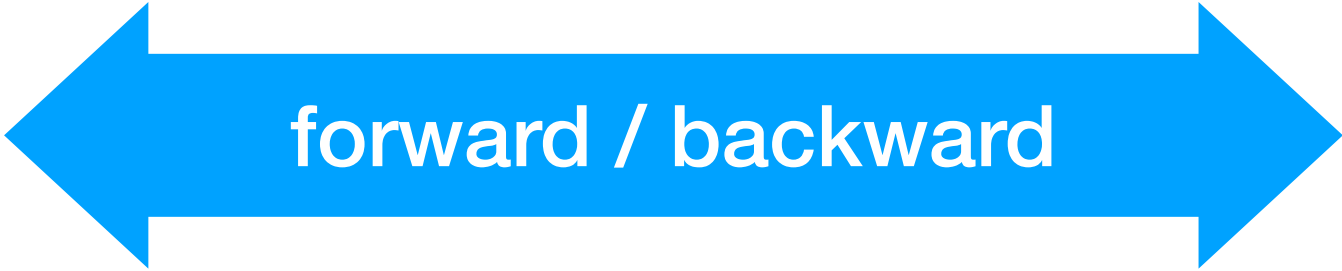
Correctness

Th. Any derivable HL triple is valid

$\{P\} \text{ skip } \{P\}$ is valid, since $\llbracket \text{skip} \rrbracket P = P \subseteq P$

Composition rule

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}}$$



forward / backward

$$\frac{\begin{array}{l} \{x = r + qy\} r := r - y \{x = r + (q + 1)y\} \\ \{x = r + (q + 1)y\} q := q + 1 \{x = r + qy\} \end{array}}{\{x = r + qy\} r := r - y; q := q + 1 \{x = r + qy\}}$$

Inlining assertions

$$\frac{\begin{array}{l} \{x = r + qy\} \ r := r - y \ \{x = r + (q + 1)y\} \\ \{x = r + (q + 1)y\} \ q := q + 1 \ \{x = r + qy\} \end{array}}{\{x = r + qy\} \ r := r - y; q := q + 1 \ \{x = r + qy\}}$$

$$\{x = r + qy\}$$

$$r := r - y;$$

$$\{x = r + (q + 1)y\}$$

$$q := q + 1$$

$$\{x = r + qy\}$$

Correctness

Th. Any derivable HL triple is valid

Proof. By induction on the derivation tree, e.g.

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}} \quad \text{We prove the conclusion is valid assuming the premises are valid}$$

$$\llbracket c_1; c_2 \rrbracket P = \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket P) \subseteq \llbracket c_2 \rrbracket R \subseteq Q$$

Conditional rule

$$\frac{\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}}$$

{true}

if $x \geq 0$ do

$\{x \geq 0\}$

 skip;

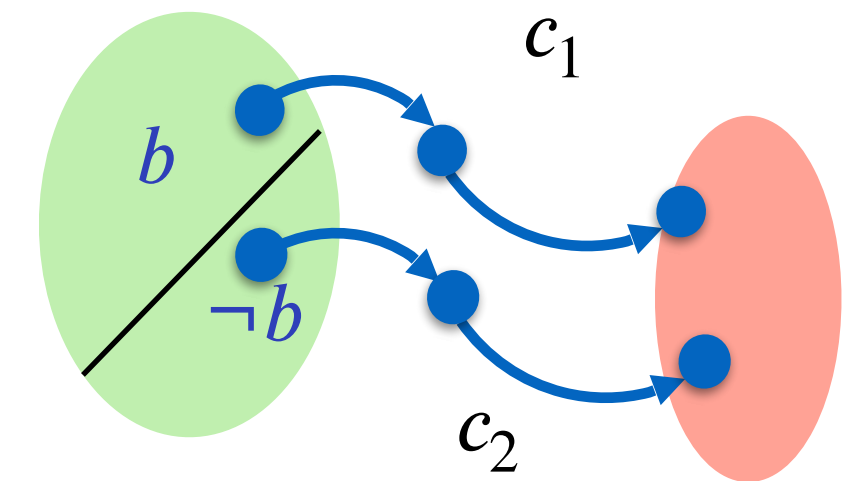
else

$\{\neg(x \geq 0)\} \equiv \{x < 0\}$

$x := -x;$

$\{x > 0\} \Rightarrow \{x \geq 0\}$

$\{x \geq 0\}$



While rule

$$\{P \wedge b\} c \{P\}$$

$$\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}$$

loop
invariant

$$\{x \geq 0\}$$

while $x > 0$ do

$x := x - 1;$

While rule

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}}$$

loop
invariant

$\{x \geq 0\}$

while $x > 0$ do

$\{x \geq 0 \wedge x > 0\}$

$x := x - 1;$

While rule

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}}$$

loop
invariant

$$\{x \geq 0\}$$

while $x > 0$ do

$$\{x \geq 0 \wedge x > 0\} \equiv \{x > 0\} \equiv \{x \geq 1\} \equiv \{x - 1 \geq 0\}$$

$x := x - 1;$

While rule

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}}$$

loop
invariant

$$\{x \geq 0\}$$

while $x > 0$ do

$$\{x \geq 0 \wedge x > 0\} \equiv \{x > 0\} \equiv \{x \geq 1\} \equiv \{x - 1 \geq 0\}$$

$x := x - 1;$

$$\{x \geq 0\}$$

While rule

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{while } b \text{ do } c \{P \wedge \neg b\}}$$

loop
invariant

$$\{x \geq 0\}$$

while $x > 0$ do

$$\{x \geq 0 \wedge x > 0\} \equiv \{x > 0\} \equiv \{x \geq 1\} \equiv \{x - 1 \geq 0\}$$

$x := x - 1;$

$$\{x \geq 0\}$$

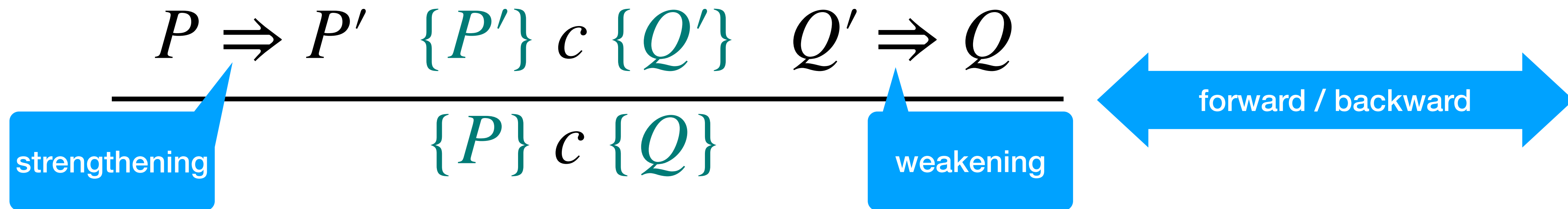
$$\{x \geq 0 \wedge x \leq 0\} \equiv \{x = 0\}$$

Invariants

Instead of reasoning about each individual iteration, we give a single correctness proof that must account for all possible loop iterations.

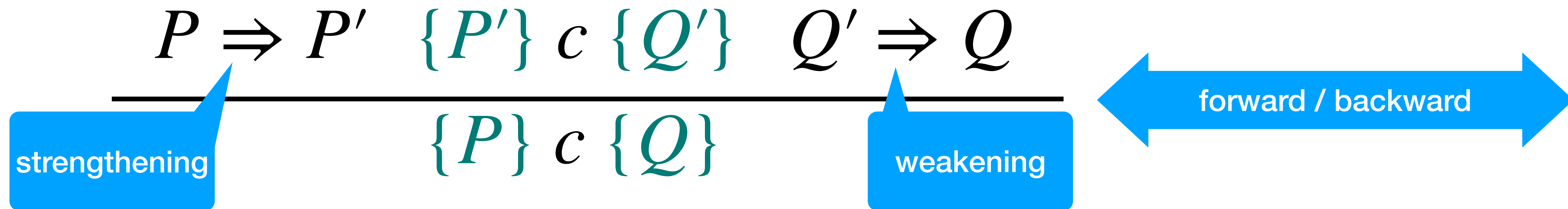
- We do not know how many times the loop will execute
- The proof must be valid for any number of iterations
- The invariant expresses a general condition that is true for every execution, but is still strong enough to guarantee the desired postcondition when the loop terminates

Consequence rule



$\{x - y < x \wedge x + y \geq 0\}$
 $n := x - y;$
 $\{n < x \wedge x + y \geq 0\}$

Consequence rule



$$\{-y < 0 \wedge x \geq 0 \wedge y \geq 0\} \Rightarrow$$
$$\{x - y < x \wedge x + y \geq 0\}$$

$n := x - y;$

$$\{n < x \wedge x + y \geq 0\}$$

Consequence rule

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}}$$

Diagram illustrating the Consequence rule with annotations:

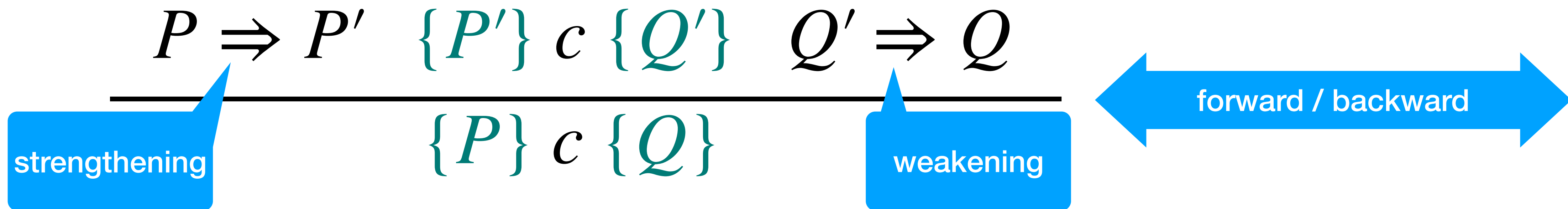
- strengthening**: Points to the antecedent $P \Rightarrow P'$.
- weakening**: Points to the consequent $Q' \Rightarrow Q$.
- forward / backward**: A large blue arrow pointing right, indicating the direction of the rule's application.

$$\begin{aligned} &\{x > 0 \wedge y > 0\} \Rightarrow \\ &\{-y < 0 \wedge x \geq 0 \wedge y \geq 0\} \Rightarrow \\ &\{x - y < x \wedge x + y \geq 0\} \end{aligned}$$

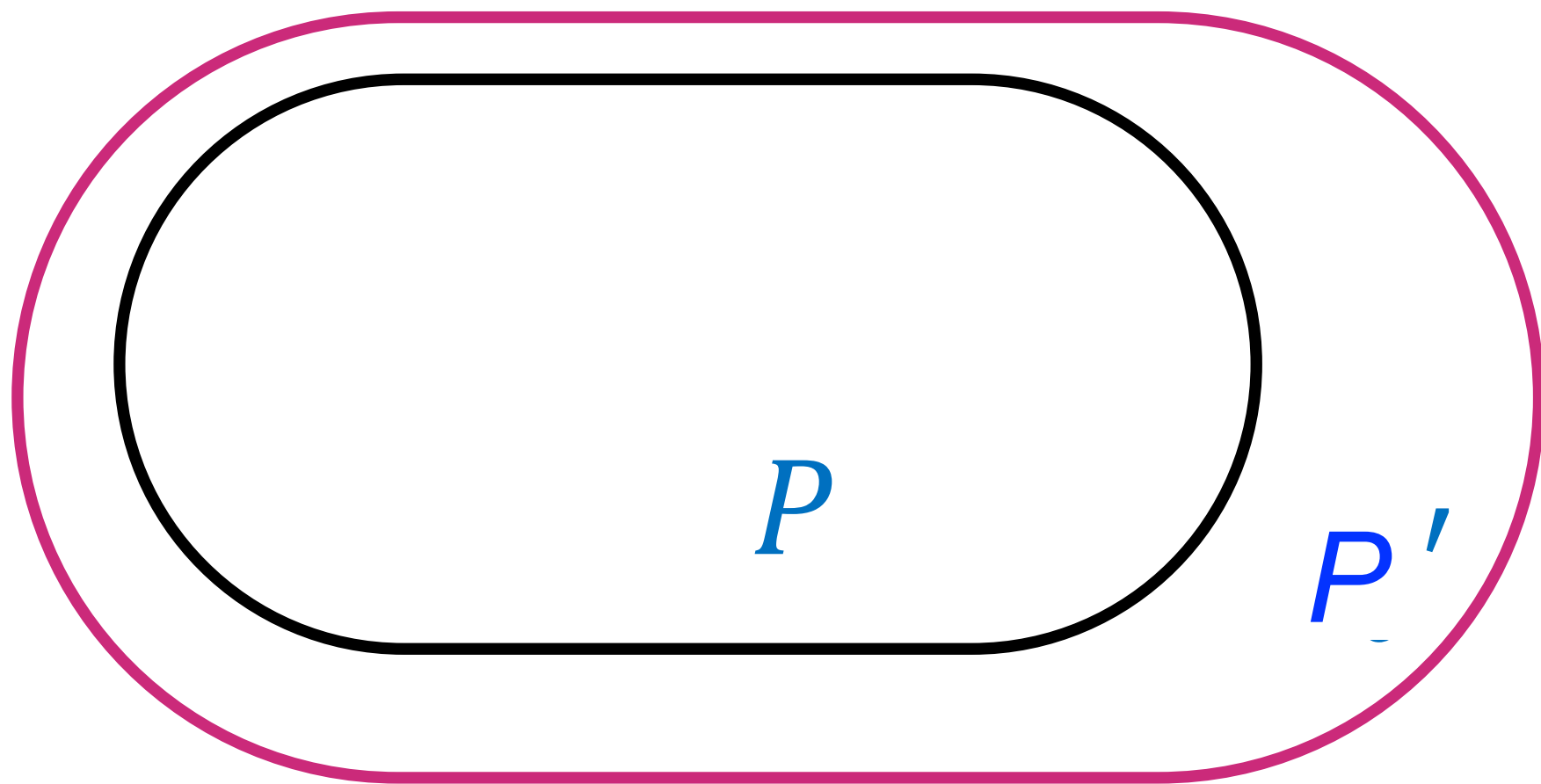
$n := x - y;$

$$\{n < x \wedge x + y \geq 0\}$$

Consequence rule



$$P \subseteq P'$$



$$\{P'\} c \{Q'\}$$

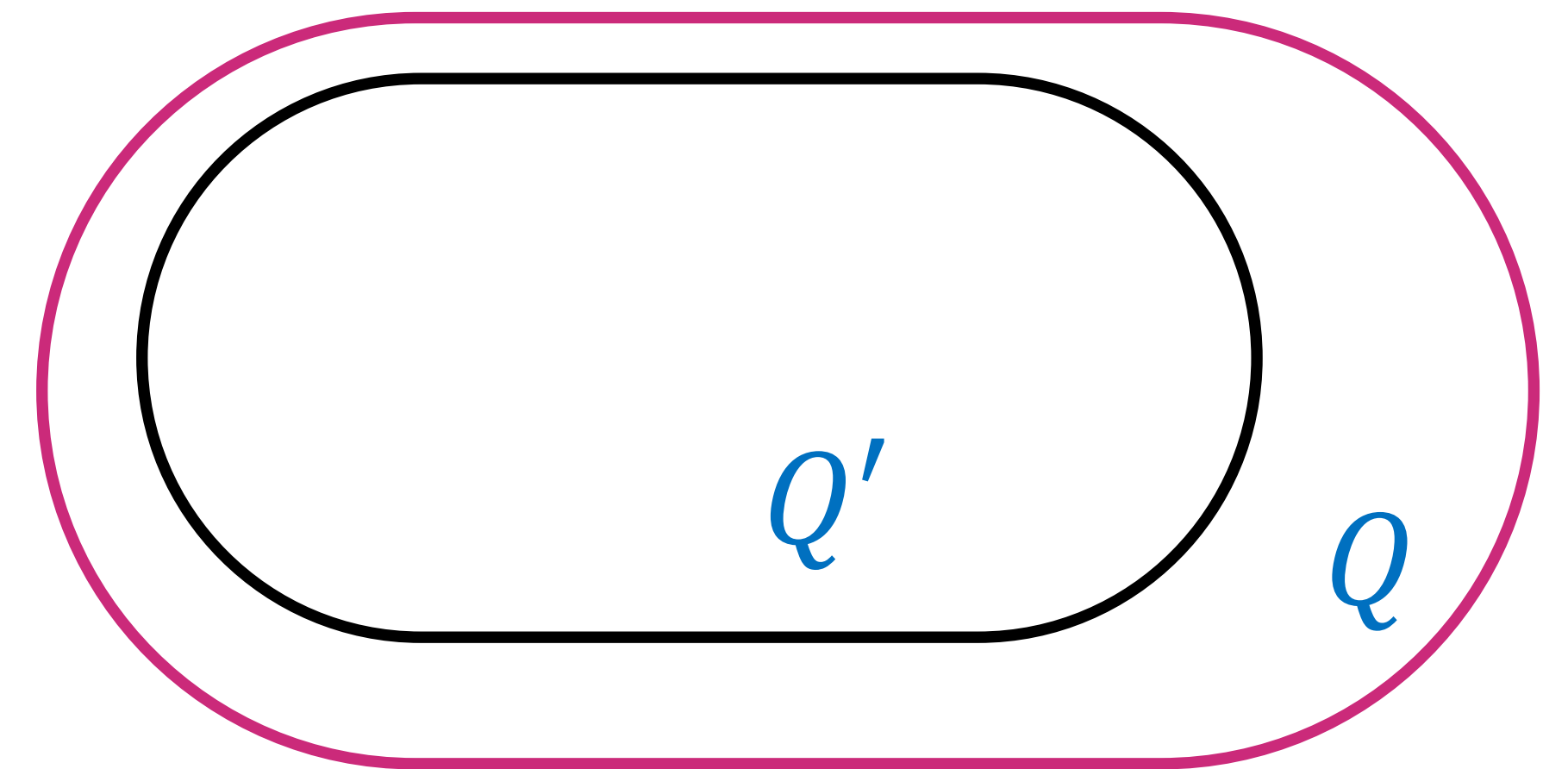
iff

$$\llbracket c \rrbracket P' \subseteq Q'$$

then

$$\llbracket c \rrbracket P \subseteq \llbracket c \rrbracket P' \subseteq Q' \subseteq Q$$

$$Q' \subseteq Q$$



The HL rules

$$\frac{}{\{P\} \text{ skip } \{P\}}$$

$$\frac{}{\{P\} x := a \{ \exists x'. P[x'/x] \wedge x = a[x'/x] \}}$$

$$\frac{}{\{Q[a/x]\} x := a \{Q\}}$$

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}}$$

$$\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}$$

$$\frac{}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}}$$

$$\{P \wedge b\} c \{P\}$$

$$\frac{}{\{P\} \text{ while } b \text{ do } c \{P \wedge \neg b\}}$$

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}}$$

Validity

A HL triple $\{P\} c \{Q\}$ is **valid** if $\llbracket c \rrbracket P \subseteq Q$

Is $\{x > 0\} x := 10x \{x > 10\}$ valid? 

Is $\{x > 0, y > 0\} x := yx \{x \geq 0\}$ valid? 

Is $\{\text{false}\} c \{Q\}$ valid? 

Is $\{P\} c \{\text{true}\}$ valid? 

Exercise

Is this rule valid in HL?

$$\frac{\{P\} \text{ } c \text{ } \{Q\}}{\{P \wedge R\} \text{ } c \text{ } \{Q \wedge R\}}$$

Exercise

Is this rule valid in HL?

$$\frac{\{P\} \text{ } c \text{ } \{Q\}}{\{P \wedge R\} \text{ } c \text{ } \{Q\}}$$

Exercise

Is this rule valid in HL?

$$\frac{\{P\} \text{ } c \text{ } \{Q\}}{\{P\} \text{ } c \text{ } \{Q \wedge R\}}$$

Exercise

Is this rule valid in HL?

$$\frac{\{P\} \text{ } c \text{ } \{Q\} \quad \{R\} \text{ } c \text{ } \{Q\}}{\{P \vee R\} \text{ } c \text{ } \{Q\}}$$

Exercise

Is this rule valid in HL?

$$\frac{\{P\} \text{ } c \text{ } \{R\} \quad \{R\} \text{ } c \text{ } \{Q\}}{\{P\} \text{ } c \text{ } \{Q\}}$$